SHIMANO BT-E6000 BMS

UART Protocol Reverse Engineering

Complete analysis of the communication protocol between

the Shimano BT-E6000 battery pack, charger (EC-E6000), and motor controller

Document date: March 16, 2026

Hardware	Renesas R2A240 (78K0 MCU) + TI BQ77PL900 (BMS IC)
	UART 3-Wire: RX, TxD, GND @ ~115200 Baud, 8N1
Protocol	Proprietary Shimano STEPS UART framing
	CRC-16/CCITT poly=0x1021 refin=true refout=true
Auth	Challenge-Response (replay-vulnerable — no nonce counter)
	Live UART captures: FROM_BATT_002, FROM_CHARGER_002, ONLINE_FIND

This document is produced for educational and interoperability research purposes. All information was obtained by passive observation of signals on hardware owned by the researcher. No proprietary firmware was extracted or modified.

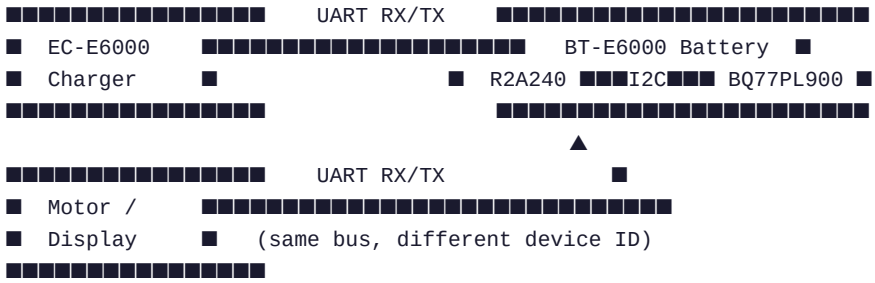
1 System Overview

The Shimano STEPS BT-E6000 is a 36 V / 11.6 Ah (418 Wh) lithium-ion battery pack designed for e-bike use. Internally, a **Renesas R2A240 60F020** (78K0-family 8-bit microcontroller) manages all external communication via UART, while a **Texas Instruments BQ77PL900** operates as the cell-level BMS in host mode, controlled by the R2A240.

1.1 Hardware Components

Component	Part Number	Role
Main MCU	Renesas R2A240 60F020	UART communication, protocol state machine
BMS IC	TI BQ77PL900	Cell monitoring, protection, auth (host mode)
Cell chemistry	Li-Ion 10S4P	36 V nominal, ~40 cells (18650)
Charger IC	Unknown (charger PCB)	CC/CV control, UART master

1.2 Communication Topology



1.3 Device Identifiers (Byte 1 of every frame)

Byte 1 value	Device	Direction
0x40	Display / Motor Controller	Transmit (to battery)
0x41	Charger (EC-E6000)	Transmit (to battery)
0x80–0x83	Battery	Transmit (response, seq mod 4)
0xC1	Battery	Header / Greeting response

2 Frame Format

Every UART message follows a consistent binary framing structure. The first byte is always **0x00**. The second byte encodes the device identity and a 2-bit sequence counter. Byte 3 carries the payload length, and the final two bytes are a CRC-16 checksum.

```
Byte offset:  0     1       2      3       4 … (4+len-1)   last-1  last
              ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
              0x00  [SEQ]  [LEN]  [TYPE]  [PAYLOAD × LEN]  [CRC_H] [CRC_L]

SEQ  = device_base | (counter & 0x03)
       e.g. battery=0x80, counter=2 → 0x82
LEN  = number of payload bytes (does NOT include 0x00, SEQ, LEN, TYPE, CRC)
TYPE = frame category (0x10=telemetry, 0x30/0x31=auth, 0x32=energy, …)
```

2.1 Known Frame Types

TYPE byte	Direction	Description
0x00 (len=0)	Both	Greeting / wake-up handshake
0x10	Battery → Charger	Telemetry (voltage, current, temp, SOC)
0x10	Charger → Battery	Polling keep-alive (all-zero payload)
0x11	Auth	Authentication sub-frame A
0x12	Auth	Authentication sub-frame B (response)
0x30	Charger → Battery	Auth challenge (12 bytes)
0x31	Battery → Charger	Auth + capabilities response
0x32	Charger → Battery	Energy/SOC query

2.2 Sequence Counter Behaviour

The second nibble of Byte 1 cycles through 0–3 (mod 4) independently for each device. The battery uses base **0x80**, charger uses **0x00**, display uses **0x40**. This allows the receiver to detect dropped frames but does *not* prevent replay attacks because the counter is not included in the authentication payload.

3 CRC Algorithm

The CRC was recovered using **CRC Reveng** from multiple captured frames. All frames use the same polynomial but with a *frame-type-dependent* initial value. This effectively means the init value is derived from the constant header bytes that Reveng treats as already processed.

```
Algorithm : CRC-16/CCITT
Polynomial: 0x1021
RefIn     : true
RefOut    : true
XorOut    : 0x0000
Covers    : bytes from offset 0 (0x00) through last payload byte inclusive
```

3.1 Init Values per Frame Type

Source	Frame type	CRC Init	Reveng command used
Battery	Auth (0x30)	0x0746	reveng -w 16 -s <auth frames>
Battery	Telemetry (0x10)	0xF46E	reveng -w 16 -s <telem frames>
Charger	Auth challenge	0xD714	reveng -w 16 -p 1021 -x 0 -s …
Charger	Capabilities (0x31)	0xEEA9	reveng -w 16 -p 1021 -x 0 -s …
Charger	Polling (0x10)	0x8683	reveng -w 16 -s <poll frames>
All	Greeting (0x00)	0x337D	reveng -w 16 -p 1021 -x 0 -s …

Python implementation (applies to polling frames, init=0x8683):

```
import crcmod

crc_fn = crcmod.mkCrcFun(0x11021, initCrc=0x8683, rev=True, xorOut=0x0000)

frame = bytes.fromhex("0000051000000000")
crc   = crc_fn(frame)
full  = frame + crc.to_bytes(2, "little")  # → 00 00 05 10 00 00 00 00 B7 2C
```

4 Authentication Protocol

Frame exchange 2 (TYPE 0x30/0x31) implements a challenge-response authentication. The charger sends a challenge; the battery (via the BQ77PL900 in host mode) produces a response. The TI BQ77PL900 datasheet documents a SHA-1-based authentication mechanism, which is consistent with what is observed here.

4.1 Observed Challenge / Response Pairs

From the original 7-session capture (SMN-TEST.pdf):

Session	Challenge (6 bytes)	Response (14 bytes, truncated)
1	D1 0E BA F0 5A 49	E3 66 F5 01 A2 40 EC 1E 61 3F 84 89 CF 68
2	0F BB F1 5B 4A D2	37 C1 D8 81 DB D2 F7 C3 F4 9E 09 9D 99 A7
3	F3 5D 4C D4 11 BD	D0 06 20 FE AE 94 28 04 E7 DC FD 50 52 11
4	D7 14 C0 F6 60 4F	CD BC C0 C2 C0 0B 3E 7A 23 8E A4 CB 82 71
5	F9 63 52 DA 17 C3	93 59 86 12 AA A0 E1 08 CD 32 C8 B1 0D EB
6	1B C7 FD 67 56 DE	DA 00 51 58 A3 9E DE 5A 2E 01 7F 40 55 94
7	C8 FE 68 57 DF 1C	76 89 1A CD 99 F2 D8 FE 37 FB 0A 1E 22 83

4.2 Challenge Structure Analysis

Careful examination of the challenge bytes from **FROM_CHARGER_002.txt** reveals that the 12-byte challenge payload is *not* cryptographically random. It consists of repeated 2- or 4-byte patterns:

```
Session 1:  9E 9E  9E 9F  F6 F6  F6 F6  9F 9F  9F 9F  ← 2-byte groups
Session 2:  9E BA  9E BA  12 3E  12 3E  BB C7  BB C7  ← repeated pairs

⇒ The nonce is produced by an LFSR or simple counter inside the charger MCU,
  NOT by a cryptographically secure RNG.
```

4.3 Why Replay Attacks Work

Because the challenge nonce is predictable and the authentication response contains no session-bound counter, a recorded challenge/response pair can be replayed indefinitely. This is the approach taken by the open-source tool **gregyedlik/BT-E60xx**, which records one authentic session and replays it to unlock the battery without Shimano hardware.

```
Replay attack flow:
  1. Record full startup sequence with original charger + battery.
  2. Store as pickle / byte array.
  3. On next power-on, play back stored bytes via UART.
  4. Battery unlocks → charge/discharge FETs enabled.

Missing protection: sequence number in auth payload, session timestamp,
                    or HMAC over nonce + timestamp.
```

4.4 Hypothesised Algorithm

The TI BQ77PL900 Application Note SLUA625 documents an **HMAC-SHA-1**-based authentication where:

- A 4-byte nonce is sent by the host (here extended to 6 bytes by Shimano)
- The BQ77PL900 computes HMAC-SHA-1(secret_key, nonce)
- The first 14 bytes of the 20-byte digest are returned (truncated MAC)
- The secret key is stored in the BQ77PL900 OTP / protected flash region

Note: The secret key has not been extracted. A flash dump of the R2A240 or decap of the BQ77PL900 would be required to confirm this hypothesis.

5 Telemetry Frame Decode (TYPE 0x10, Battery → Charger)

The main recurring data frame is 27 bytes long (header + 22 payload + 2 CRC). Based on FROM_BATT_002.txt, which contains clean, high-sampling-rate data recorded during a full charge cycle, the following field map has been derived:

Offset	Bytes	Type	Field	Notes / Scaling
0	1	const	0x00	Frame start sentinel
1	1	uint8	SEQ	0x80–0x83 (base 0x80 + counter mod 4)
2	1	uint8	LEN	0x16 = 22 (payload bytes)
3	1	uint8	TYPE	0x10 = telemetry
4	1	flags	FLAGS	Bit flags (observed: 0x00, 0x80, 0x20)
5	1	uint8	MODE	0x02=startup, 0x03=charging active
6–8	3	uint8[3]	RESERVED	Always 0x00 in captures
9–10	2	uint16-LE	PACK_MV	Pack voltage in millivolts 0x9FFD = 40957 mV = 40.96 V 0xA246 = 41542 mV = 41.54 V
11	1	uint8	I_RAW	Charge current (raw) — scaling TBD Rises during CC phase
12	1	const	0x20	Separator / fixed field
13	1	uint8	I_TARGET?	Target current or cell delta — TBD
14	1	uint8	TEMP_PACK	Pack temperature in °C 0x1F=31°C, 0x20=32°C
15	1	uint8	TEMP_1	Sensor 1 temperature in °C
16	1	uint8	TEMP_2	Sensor 2 temperature in °C
17	1	uint8	TEMP_3	Sensor 3 temperature in °C (rises last)
18	1	uint8	CELL_IDX	0x5C/0x5D — SOC range or cell index
19	1	uint8	STATUS	0x4D/0x4E alternates — phase flag?
20–24	5	uint8[5]	RESERVED	All zeros in current captures
25–26	2	uint16	CRC	CRC-16/CCITT, init=0xF46E

5.1 Voltage Progression During Charge

Pack voltage steadily increases — confirming the field decode:

```
Frame  1: PACK_MV = 0x9FFD = 40.96 V  (start of session)
Frame  8: PACK_MV = 0xA1E6 = 41.44 V
Frame 16: PACK_MV = 0xA211 = 41.45 V
Frame 32: PACK_MV = 0xA23F = 41.59 V
Frame 35: PACK_MV = 0xA246 = 41.54 V  (end of capture)

10S pack → 41.54 V / 10 = 4.154 V per cell (mid-CC phase, expected ✓)
```

5.2 Temperature Sensors

```
Sensor 1 (offset 15): 0x18 = 24°C  → stable throughout
Sensor 2 (offset 16): 0x18 = 24°C  → stable throughout
Sensor 3 (offset 17): 0x18 → 0x19  → slight rise at end of capture
Pack    (offset 14): 0x1F → 0x20   → ambient + modest self-heating
```

Values appear to be direct Celsius with no offset (consistent with 24°C room).

5.3 MODE Byte (offset 5)

0x02 → Connection / initialisation phase (first 1–2 frames)
0x03 → Active charging (all subsequent frames)

Observed transition in FROM_BATT_002: frame 1 = 0x02, frames 2+ = 0x03

6 Charger Polling Frames (Charger → Battery)

The charger sends a fixed 4-frame cycle to poll the battery. All four frames are entirely static — no payload data changes between cycles. Their sole purpose appears to be a keep-alive heartbeat that triggers the battery to respond with a telemetry frame.

```
00 00 05 10 00 00 00 00  B7 2C   ← SEQ=0x00
00 01 05 10 00 00 00 00  62 B3   ← SEQ=0x01
00 02 05 10 00 00 00 00  0C 1B   ← SEQ=0x02
00 03 05 10 00 00 00 00  D9 84   ← SEQ=0x03
```

`CRC-16/CCITT (init=0x8683) — verified with CRC Reveng.`

Verified from ONLINE_FIND (bike motor controller recording) that the same polling frame format is used when the battery is connected to the motor controller, confirming the bus protocol is identical for both charger and drive-unit contexts.

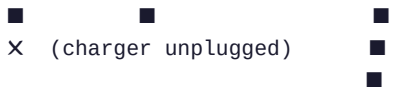
7 Complete Session Flow

A full charging session consists of the following ordered exchanges:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  PHASE 1 — GREETING                                                        ■
■  Charger  →  Battery:  00 41 00 F9 50                                      ■
■  Battery  →  Charger:  00 C1 00 35 DC                                      ■
■  (Wake-up handshake, static, no payload)                                   ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  PHASE 2 — AUTHENTICATION  (TYPE 0x30 / 0x31)                              ■
■  Charger  →  Battery:  Challenge (12-byte LFSR nonce)                      ■
■  Battery  →  Charger:  Response  (14-byte truncated MAC)                   ■
■  (Repeated once; both frames vary each power cycle)                        ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  PHASE 3 — CAPABILITIES EXCHANGE  (TYPE 0x31 / 0x0B)                       ■
■  Battery  →  Charger:  9F 01 A9 01 01 00 02 00 1F 00 05 …                  ■
■  (Serial number?, capacity, cell count — partially decoded)                ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  PHASE 4 — CYCLIC TELEMETRY  (TYPE 0x10, repeating)                        ■
■  Charger  →  Battery:  Poll  (4-frame cycle, all zeros payload)            ■
■  Battery  →  Charger:  Telemetry (voltage, current, temp, status)          ■
■  (Runs continuously until charger is unplugged)                            ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

7.1 Sequence Diagram

```
Charger                    Battery
   ■                          ■
   ■■■■■■ 00 41 00 F9 50 ■■■■■■■■  Greeting
   ■■■■■■ 00 C1 00 35 DC ■■■■■■■■  Greeting ACK
   ■                          ■
   ■■■■■■ AUTH Challenge ■■■■■■■■■  TYPE 0x30
   ■■■■■■ AUTH Response ■■■■■■■■■■  TYPE 0x31
   ■■■■■■ Capabilities Req. ■■■■■■  TYPE 0x0B
   ■■■■■■ Capabilities Resp. ■■■■■  Battery info
   ■                          ■
   ■■■■■■ Poll  SEQ=0 ■■■■■■■■■■■■  ■
   ■■■■■■ Telemetry SEQ=0 ■■■■■■■■  ■
   ■■■■■■ Poll  SEQ=1 ■■■■■■■■■■■■  ■  Repeating
   ■■■■■■ Telemetry SEQ=1 ■■■■■■■■  ■  cycle
   ■■■■■■ Poll  SEQ=2 ■■■■■■■■■■■■  ■
   ■■■■■■ Telemetry SEQ=2 ■■■■■■■■  ■
   ■■■■■■ Poll  SEQ=3 ■■■■■■■■■■■■  ■
   ■■■■■■ Telemetry SEQ=3 ■■■■■■■■
```


(charger unplugged)

8 Open Questions & Next Steps

8.1 Current Measurement Correlation

Fields at offsets 11 and 13 in the telemetry frame are believed to encode charge current but their scaling factor has not been confirmed. The recommended approach is to measure actual current in-line with a calibrated shunt or clamp meter while simultaneously logging UART frames with timestamps.

```
Measurement plan:
  - Insert calibrated ammeter in series on the (+) charging line
  - Log UART + ammeter reading every ~5 seconds
  - Cover all charge phases: pre-charge, CC, CV, trickle
  - Record Byte[11] and Byte[13] vs. measured Amps

Hypothesis A:  I [A] = Byte[11] / 30    (0x68=104 → 3.47 A)
Hypothesis B:  I [A] = Byte[11] × 0.05  (0x68=104 → 5.20 A)
Hypothesis C:  I [mA]= Byte[11] × 32    (0x68=104 → 3328 mA = 3.33 A)
```

8.2 Capabilities Frame Decode (TYPE 0x31)

The first authentication response contains battery metadata. Partially decoded:

```
Raw:  9F 01  A9 01  01 00  02 00  1F 00  05 7F

0x019F = 415 → likely capacity in some unit (418 Wh ≈ 415?)
0x01A9 = 425 → unknown
0x0001 = 1   → unknown flag
0x0002 = 2   → charge mode?
0x001F = 31  → temperature?
0x0005 = 5   → cell count group?
0x7F   = 127 → unknown or partial CRC
```

8.3 Flash Dump (for auth key recovery)

To fully reverse-engineer the authentication algorithm and extract the secret key, a flash dump of the Renesas R2A240 is required.

Method	Difficulty	Cost	Notes
UART Bootloader (FLMD0 pin)	Easy	~3 €	May fail if security bit set
Renesas E2 Lite clone	Easy	~30 €	Official debug interface
ChipWhisperer Nano (glitch)	Medium	~50 €	If security bit is set
Decap + optical readout	Hard	200+ €	Destructive, last resort

8.4 Related Open-Source Projects

Project	URL	Approach
gregyedlik/BT-E60xx	github.com/gregyedlik/BT-E60xx	UART replay — works today
michielvg/Shimano_BT-E6000_BMS	github.com/michielvg/…	Protocol analysis (this source)
rolandvs/shimano	github.com/rolandvs/shimano	E-Tube firmware exploration

9 Quick Reference Card

9.1 CRC Python Snippet

```
import crcmod

# Choose init based on frame source:
INIT = {
    "charger_poll":   0x8683,
    "batt_telemetry": 0xF46E,
    "auth_challenge": 0xD714,
    "greeting":       0x337D,
}

def make_crc(init_val):
    return crcmod.mkCrcFun(0x11021, initCrc=init_val, rev=True, xorOut=0)

def frame_crc(data: bytes, init_val: int) -> bytes:
    fn  = make_crc(init_val)
    val = fn(data)
    return val.to_bytes(2, "little")
```

9.2 Minimal Charger Emulator (Python)

```
import serial, time

POLL = [
    bytes.fromhex("000005100000000062B3"),   # SEQ 0
    bytes.fromhex("000105100000000062B3"),   # SEQ 1 — replace with correct CRC
]

ser = serial.Serial("/dev/ttyUSB0", 115200, timeout=0.1)

# Step 1: send greeting
ser.write(bytes.fromhex("004100F950"))
time.sleep(0.1)

# Step 2: replay stored auth frames (gregyedlik approach)
# ser.write(stored_auth_challenge)

# Step 3: poll loop
seq = 0
while True:
    ser.write(POLL[seq % 2])
    resp = ser.read(64)
    if resp:
        v_mv = int.from_bytes(resp[9:11], "little")
        print(f"Pack voltage: {v_mv/1000:.3f} V  Temp: {resp[14]}°C")
    seq += 1
    time.sleep(0.25)
```

9.3 Frame Field Summary

Field	Offset	Decode
Pack voltage	9–10	uint16-LE in mV ÷ 1000 = Volts
Charge current	11	Raw uint8 — scaling TBD (see §8.1)
Pack temperature	14	Direct °C, no offset
Sensor 1	15	Direct °C

Field	Offset	Decode
Sensor 2	16	Direct °C
Sensor 3	17	Direct °C
MODE	5	0x02=init 0x03=charging
CRC	25–26	CRC-16/CCITT, init=0xF46E, little-endian